

Handling time series data

December 15, 2024

Time Series Data

Time series data: Time series data is data recorded over a period of time. It can be described as a collection of values a collection of observations on the values that a variable takes at various points in time.

Examples of time series data 1. Stocks 2. Weather 3. Sales data 5. Sensor readings

Crosssectional data : Data from one or more variables that were collected simultaneously

Time series analysis: Methods for analyzing time series data for the purpose of extracting meaningful information

The key concept in time series data is time. We need to be able to know how to handle time in the time series data. Python has a module called datetime, used to handle the data.

Datetime Module

The python datetime module has classes that are used for handling dates and time.

Current date

To get the current date we will use the today function in the datetime module as follows

```
[1]: from datetime import date
      today= date.today()
      print(today)
```

2024-12-14

Getting the year, month and day aspect of the datetime module We can use the year, month and day attributes of the date object to extract the year, month and day from a date as follows

```
[2]: from datetime import date
      today= date.today()
      year=today.year
      month=today.month
      day=today.day
      print(f"Year:{year}\nMonth:{month}\nDay:{day}")
```

Year:2024

Month:12

Day:14

The datetime object

Datetime stores both the date and time down to the microsecond

```
[3]: from datetime import datetime
now=datetime.now()
print(now)
```

2024-12-14 09:23:12.897130

Timedelta: timedelta represents the temporal difference between two datetime objects:

```
[4]: delta = datetime(2011, 1, 7) - datetime(2008, 6, 24, 8, 15)
print(delta)
#days
print(delta.days)
```

926 days, 15:45:00

926

You can add (or subtract) a timedelta or multiple thereof to a datetime object to get a new shifted object

```
[5]: from datetime import timedelta
start=datetime(2024,1,7)
start=start + timedelta(12) #Adds 12 to the date component of the time
print(start)
```

2024-01-19 00:00:00

Types in datetime module

1. date: Store calendar date (year, month, day) using the Gregorian calendar
2. time: Store time of day as hours, minutes, seconds, and microseconds
3. datetime: Stores both date and time
4. timedelta: Represents the difference between two datetime values (as days, seconds, and microseconds)
5. tzinfo: Base type for storing time zone information

```
[6]: #converting to strings
stamp = datetime(2011, 1, 3)
print(type(stamp))
convert_string=str(stamp)
print(type(convert_string))
print(convert_string)
```

<class 'datetime.datetime'>

<class 'str'>

2011-01-03 00:00:00

```
[7]: #Formatting the string object
format_stamp=stamp.strftime('%Y-%m-%d')
```

```
print(type(format_stamp))
print(format_stamp)
```

```
<class 'str'>
2011-01-03
```

Datetime format specification (ISO C89 compatible)

1. %Y: Four-digit year
2. %y: Two-digit year
3. %m: Two-digit month [01, 12]
4. %d: Two-digit day [01, 31]
5. %H: Hour (24-hour clock) [00, 23]
6. %I: Hour (12-hour clock) [01, 12]
7. %M: Two-digit minute [00, 59]
8. %S: Second [00, 61] (seconds 60, 61 account for leap seconds)
9. %w: Weekday as integer [0 (Sunday), 6]
10. %U: Week number of the year [00, 53]; Sunday is considered the first day of the week, and days before the first Sunday of the year are “week 0”
11. %W: Week number of the year [00, 53]; Monday is considered the first day of the week, and days before the first Monday of the year are “week 0”
12. %z: UTC time zone offset as +HHMM or -HHMM; empty if time zone naive
13. %F: Shortcut for %Y-%m-%d (e.g., 2012-4-18)
14. %D: Shortcut for %m/%d/%y (e.g., 04/18/12)

The string formats can be used for converting strings to dates using the datetime.strptime function as follows

```
[8]: dates='2011-01-03'
print(type(dates))
print(dates)
new_dates=datetime.strptime(dates, '%Y-%m-%d')
print(new_dates)
print(type(new_dates))
```

```
<class 'str'>
2011-01-03
2011-01-03 00:00:00
<class 'datetime.datetime'>
```

Preprocessing time series data

Date Ranges, Frequencies, and Shifting

pandas has a suite of standard time series frequencies and tools for resampling, inferring frequencies, and generating fixed-frequency date ranges. For example, you can convert the sample time series to be fixed daily frequency by calling resample

Generating Date ranges pandas.date_range is responsible for generating a DatetimeIndex with an indicated length according to a particular frequency:

```
[43]: index = pd.date_range('2024-04-01', '2027-06-01')
index
```

```
[43]: DatetimeIndex(['2024-04-01', '2024-04-02', '2024-04-03', '2024-04-04',
                    '2024-04-05', '2024-04-06', '2024-04-07', '2024-04-08',
                    '2024-04-09', '2024-04-10',
                    ...,
                    '2027-05-23', '2027-05-24', '2027-05-25', '2027-05-26',
                    '2027-05-27', '2027-05-28', '2027-05-29', '2027-05-30',
                    '2027-05-31', '2027-06-01'],
                    dtype='datetime64[ns]', length=1157, freq='D')
```

By default, `date_range` generates daily timestamps. If you pass only a start or enddate, you must pass a number of periods to generate. The code below will generate 10 periods.

```
[44]: index_period=pd.date_range('2024-04-01', '2027-06-01', periods=10)
index_period
```

```
[44]: DatetimeIndex(['2024-04-01 00:00:00', '2024-08-07 10:40:00',
                    '2024-12-13 21:20:00', '2025-04-21 08:00:00',
                    '2025-08-27 18:40:00', '2026-01-03 05:20:00',
                    '2026-05-11 16:00:00', '2026-09-17 02:40:00',
                    '2027-01-23 13:20:00', '2027-06-01 00:00:00'],
                    dtype='datetime64[ns]', freq=None)
```

The start and end dates define strict boundaries for the generated date index. For example, if you wanted a date index containing the last business day of each month, you would pass the 'BME' frequency as follows

```
[46]: business_index=pd.date_range('2024-01-01', '2024-12-01', freq='BME')
business_index
```

```
[46]: DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-29', '2024-04-30',
                    '2024-05-31', '2024-06-28', '2024-07-31', '2024-08-30',
                    '2024-09-30', '2024-10-31', '2024-11-29'],
                    dtype='datetime64[ns]', freq='BME')
```

Data ranges frequencies and shifting

Some applications will require the time series data to be in a specific frequencies such as daily, monthly, hourly or every fifteen minutes. Pandas has tools for resampling, inferring frequencies, and generating fixed frequency data ranges.

Resampling Resampling refers to the process of converting a time series from one frequency to another. Aggregating higher frequency data to lower frequency is called downsampling, while converting lower frequency to higher frequency is called upsampling.

When converting from a low frequency to a higher frequency, no aggregation is needed.

```
[24]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
data=pd.read_csv("yahoo_stock.csv")
data.head()
```

```
[24]:
```

	Date	High	Low	Open	Close \
0	2015-11-23	2095.610107	2081.389893	2089.409912	2086.590088
1	2015-11-24	2094.120117	2070.290039	2084.419922	2089.139893
2	2015-11-25	2093.000000	2086.300049	2089.300049	2088.870117
3	2015-11-26	2093.000000	2086.300049	2089.300049	2088.870117
4	2015-11-27	2093.290039	2084.129883	2088.820068	2090.110107

	Volume	Adj Close
0	3.587980e+09	2086.590088
1	3.884930e+09	2089.139893
2	2.852940e+09	2088.870117
3	2.852940e+09	2088.870117
4	1.466840e+09	2090.110107

```
[25]: data['Date'] = pd.to_datetime(data['Date'])
```

```
[26]: #set index to date
data.set_index(['Date'], inplace=True)
```

```
[27]: data.head()
```

```
[27]:
```

	High	Low	Open	Close	Volume \
Date					
2015-11-23	2095.610107	2081.389893	2089.409912	2086.590088	3.587980e+09
2015-11-24	2094.120117	2070.290039	2084.419922	2089.139893	3.884930e+09
2015-11-25	2093.000000	2086.300049	2089.300049	2088.870117	2.852940e+09
2015-11-26	2093.000000	2086.300049	2089.300049	2088.870117	2.852940e+09
2015-11-27	2093.290039	2084.129883	2088.820068	2090.110107	1.466840e+09

	Adj Close
Date	
2015-11-23	2086.590088
2015-11-24	2089.139893
2015-11-25	2088.870117
2015-11-26	2088.870117
2015-11-27	2090.110107

```
[37]: data_resample=data[['High']].resample('h').ffill()
data_resample.head()
```

```
[37]:
```

Date	High
2015-11-23 00:00:00	2095.610107
2015-11-23 01:00:00	2095.610107
2015-11-23 02:00:00	2095.610107
2015-11-23 03:00:00	2095.610107
2015-11-23 04:00:00	2095.610107

The `ffill()` function is used to fill the missing values that results from the resampling. `ffill()` function in pandas stands for “forward fill” and is used to propagate the last valid value forward to fill missing values (NaNs) in a DataFrame or Series

```
[39]: #Downsampling
avr_month=data[['High']].resample('ME').mean()
avr_month.head()
```

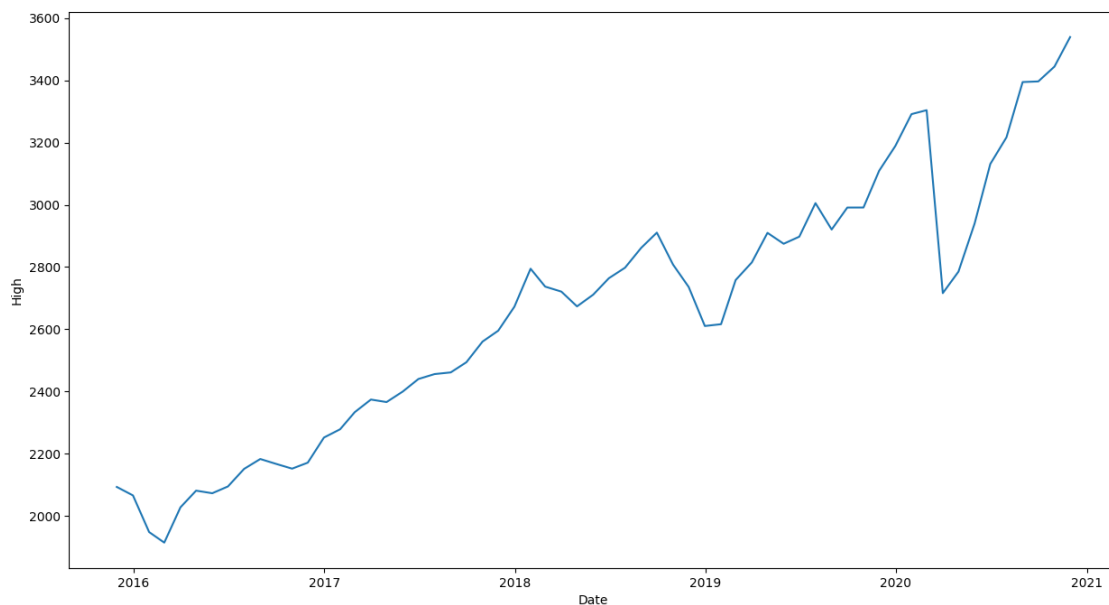
```
[39]:
```

Date	High
2015-11-30	2093.676300
2015-12-31	2066.437146
2016-01-31	1948.652903
2016-02-29	1914.799304
2016-03-31	2028.116132

We can visualize the same using matplotlib

```
[40]: plt.figure(figsize=(15,8))
sns.lineplot(x=avr_month.index, y=avr_month['High'])
```

```
[40]: <Axes: xlabel='Date', ylabel='High'>
```



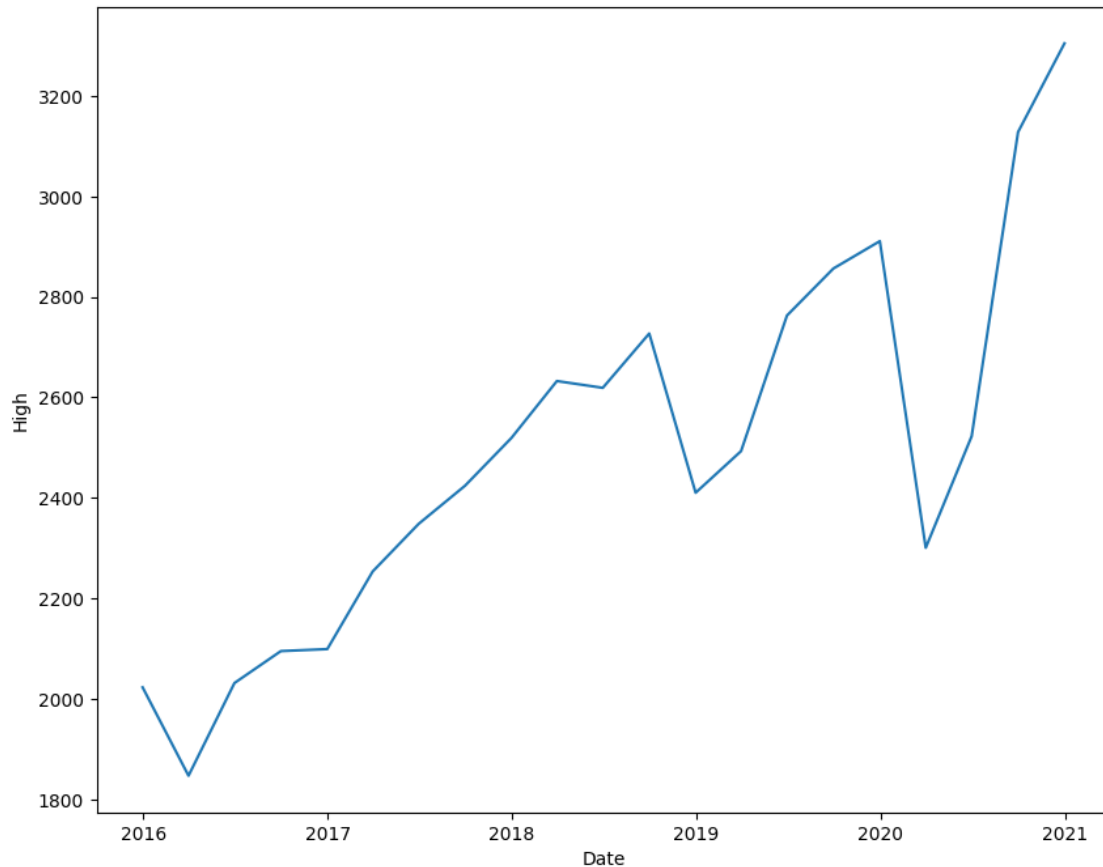
We can use the results of the resampling to get the particular information on the particular frequencies. For example we can try to find out the lowest High value in bry resampling to the quarter fequency and culculating the minimum value. This will output the minimum value for each of the quaters.

```
[41]: quarter_low=data[["High"]].resample("QE").min()  
      print(quarter_low)
```

Date	High
2015-12-31	2022.900024
2016-03-31	1847.000000
2016-06-30	2031.449951
2016-09-30	2095.050049
2016-12-31	2099.070068
2017-03-31	2253.580078
2017-06-30	2348.260010
2017-09-30	2424.280029
2017-12-31	2519.439941
2018-03-31	2632.649902
2018-06-30	2619.139893
2018-09-30	2727.260010
2018-12-31	2410.340088
2019-03-31	2493.139893
2019-06-30	2763.070068
2019-09-30	2856.669922
2019-12-31	2911.129883
2020-03-31	2300.729980
2020-06-30	2522.750000
2020-09-30	3128.439941
2020-12-31	3304.929932

```
[35]: plt.figure(figsize=(10,8))  
      sns.lineplot(x=quarter_low.index, y=quarter_low["High"])
```

```
[35]: <Axes: xlabel='Date', ylabel='High'>
```



Shifting (Leading and Lagging) Data

“Shifting” refers to moving data backward and forward through time. Both Series and DataFrame have a shift method for doing naive shifts forward or backward, leaving the index unmodified:

Lagging: Is moving the data backward Leading: Is moving the data forward

Shifting is useful when comparing the current records with the previous records (lagging). Or current records with future (forecasted) records (leading)

```
[50]: ts = pd.Series(np.random.randn(4), index=pd.date_range('1/1/2024', periods=4,
    ↪ freq='ME'))
ts
```

```
[50]: 2024-01-31    -0.210053
      2024-02-29    -0.375219
      2024-03-31     1.011759
      2024-04-30     0.874381
      Freq: ME, dtype: float64
```

```
[51]: ts.shift(2)
```



```
[51]: 2024-01-31      NaN
      2024-02-29      NaN
      2024-03-31   -0.210053
      2024-04-30   -0.375219
      Freq: ME, dtype: float64
```

```
[52]: ts.shift(-2)
```

```
[52]: 2024-01-31    1.011759
      2024-02-29    0.874381
      2024-03-31      NaN
      2024-04-30      NaN
      Freq: ME, dtype: float64
```

When we shift like this, missing data is introduced either at the start or the end of the time series. Because naive shifts leave the index unmodified, some data is discarded. Thus if the frequency is known, it can be passed to shift to advance the timestamps instead of simply the data:

```
[53]: ts.shift(2, freq='ME')
```

```
[53]: 2024-03-31   -0.210053
      2024-04-30   -0.375219
      2024-05-31    1.011759
      2024-06-30    0.874381
      Freq: ME, dtype: float64
```

Time zone handling